AICW NLP COMPANION

Natural Language Processing

From words to vectors to memory: reading, tagging, and modelling human language

Chapter 1 (NLP) · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. Five coloured boxes guide you through this chapter. A **blue** box gives you the idea in plain words. An **amber** box gives you a real-life picture of that idea. A **green** box solves a real example, one step at a time, with real numbers. A **red** box flags a common mistake. A **purple** box gives you the one line to remember. Read them in that order and you will never meet a symbol that was not explained first.

1 The big idea: what is NLP?

Natural Language Processing, or **NLP**, is the part of computer science that teaches machines to work with human language. It sits where three fields meet: computer science, linguistics, and artificial intelligence. A machine running NLP can read a sentence, pull out its meaning, and sometimes write a sentence back.

☞ The intuition

Think of NLP as teaching a computer to do what you do when you read a text message: figure out what the words mean, who is talking, and what they want.

★ Everyday picture

You have a friend who speaks three languages and is a fast reader. Give them a letter and they can summarise it, answer questions about it, and reply in kind. NLP tries to build that friend out of code.

NLP has two engines working inside it, and every task in this chapter is really an instance of one or the other.

NLU *Natural Language Understanding.* The machine reads text and pulls out meaning: what is the topic, what does the writer want (their *intent*), and how do they feel (their *sentiment*). Chatbots and virtual assistants rely on NLU to know what you just asked.

NLG *Natural Language Generation.* The machine goes the other way: it turns data or an idea into human-like text or speech. A weather app that writes "expect light rain this afternoon" from raw sensor numbers is doing NLG.

✎ Worked example: a one-line chatbot exchange

A user types: *"Cancel my flight to Delhi tomorrow."* Walk through what the system does.

Step 1. Read the words. Tokenize the sentence: *Cancel, my, flight, to, Delhi, tomorrow.*

Step 2. Find the intent (NLU). The verb *cancel* plus the object *flight* points to intent = `CANCEL_BOOKING`.

Step 3. Find the details (NLU). *Delhi* is a place; *tomorrow* is a date. These are the slots the system needs to act.

Step 4. Act. The backend looks up the booking that matches those slots.

Step 5. Reply (NLG). The system turns the result into a sentence: "Your flight to Delhi tomorrow has been cancelled."

So one short message uses NLU to understand and NLG to answer.

Where this shows up in ML. Every large language model is doing NLU and NLG in one pass: it reads your prompt (NLU) and writes a reply (NLG) using the same underlying network, just run in two directions of the same task.

✔ Key takeaway

NLP splits into two jobs: NLU reads meaning out of text, NLG writes meaning back into text. Almost every tool in this chapter is a building block for one of the two.

2 Where NLP shows up: a quick applications tour

NLP is not one product; it is a toolbox used across many products. The table below groups the common ones by the job they do.

Application	What it uses NLP for
Machine translation	Turning text from one language into another
Chatbots & virtual assistants	Understanding a request and replying in words
Sentiment analysis	Deciding if a review or post is positive, negative, or neutral
Spam filtering	Deciding if an email is unwanted (junk) or wanted
Voice assistants	Turning speech into text, then acting on the text
Summarisation	Compressing a long document into a few sentences
Recommendation	Reading reviews or descriptions to suggest similar items

✔ Key takeaway

If a product reads, writes, listens to, or summarises human language, there is NLP running underneath it.

3 Getting your tools ready: NLTK

NLTK (Natural Language Toolkit) is a Python library built for NLP. It ships with ready-made word lists, grammar helpers, and over 50 text collections (*corpora*) you can practise on. Install it, then download its data once:

```
pip install nltk
```

```
import nltk
nltk.download()
```

The second command opens a small downloader so NLTK can fetch the datasets and models each function needs. You only run it once per machine.

✗ Watch out

Skipping `nltk.download()` is the single most common NLTK error. If a function complains it cannot find a "resource" or a "corpus", this step was missed.

4 Tokenization: breaking text into pieces

A computer cannot work on a whole paragraph at once. The first job of any NLP pipeline is **tokenization**: cutting text into smaller pieces called *tokens* — usually words or sentences.

☞ The intuition

Tokenization is just chopping a sentence into its Lego bricks: the words. Once you have the bricks, every later step (counting words, tagging them, finding names) has something concrete to work with.

★ Everyday picture

Think of a shopping list written as one long line: "milk eggs bread butter". Before you can check items off, you split it into separate items. Tokenization does that splitting for text.

NLTK's `tokenize` module gives you two matching tools:

word_tokenize()
: Splits a sentence into word tokens.

sent_tokenize()
: Splits a paragraph into sentence tokens.

✎ Worked example: splitting a sentence into words

Take the sentence: *"Natural language processing is amazing!"*

Step 1. Write down the input. One string: `"Natural language processing is amazing!"`

Step 2. Call the tokenizer.

```
from nltk.tokenize import word_tokenize
word_tokenize("Natural language processing is amazing!")
```

Step 3. Read the output list. `['Natural', 'language', 'processing', 'is', 'amazing', '!']`

Step 4. Check the details. The tokenizer kept the exclamation mark as its own token — punctuation gets tokenized too, not thrown away.

Six tokens came out of one string: five words and one punctuation mark.

Where this shows up in ML. Modern language models use a cousin of this idea called *subword tokenization* (e.g. Byte-Pair Encoding). Instead of always splitting on whole words, it splits rare words into smaller pieces so the model never meets a completely unknown word.

✗ Watch out

Naively splitting on spaces breaks on punctuation ("amazing!" would stay as one token) and on contractions ("don't"). Use a real tokenizer, not `text.split(" ")`.

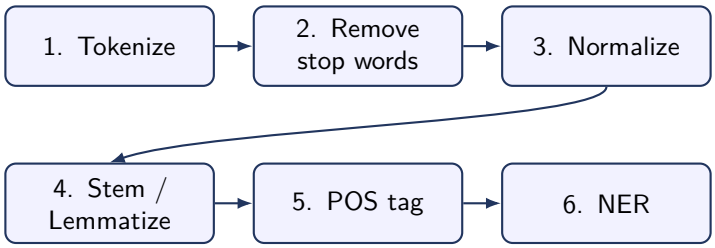


Figure 1: The six-step text preprocessing pipeline. Read left to right, then continue on row two: every later step depends on tokens produced by the first step.

✔ Key takeaway

Tokenization turns one long string into a list of tokens. Every later NLP step — counting, tagging, tagging entities — works on that list, not on the raw string.

5 Removing stop words: cutting the noise

Stop words are the small, very common words that carry little meaning on their own: *the, is, in, for, where, when, to, at.* For many tasks (like counting which topic words matter) they only add noise.

☞ The intuition

If you are trying to find out what a document is *about*, words like "the" and "is" show up everywhere and tell you nothing special. Removing them lets the meaningful words stand out.

★ Everyday picture

Picture skimming a news headline for the important nouns and verbs — you naturally skip over "the" and "a" without even noticing. Stop-word removal makes a computer do that same skipping on purpose.

✎ Worked example: removing stop words from a sentence

Take the tokens from Section 4: `['Natural', 'language', 'processing', 'is', 'amazing', '!']`.

Step 1. List the tokens. Six tokens, as tokenized earlier.

Step 2. Check each token against NLTK's English stop-word list. The word `'is'` is on that list; the rest are not (nor is the punctuation mark, which we handle separately).

Step 3. Drop the matches. Remove `'is'`.

Step 4. Read the result. `['Natural', 'language', 'processing', 'amazing', '!']`

One filler word left; the content words survived untouched.

Where this shows up in ML. Classic bag-of-words classifiers (spam filters, simple sentiment models) remove stop words before counting, so the counts reflect topic words, not filler. Transformer models like BERT skip this step — they read every word, filler included, because the surrounding words give the filler words context too.

✗ Watch out

Do not remove stop words before tasks that need word order or grammar, such as part-of-speech tagging or translation — "not" is a stop word in some lists, and dropping it can flip a sentence's meaning.

✔ Key takeaway

Stop-word removal trims common, low-information words. It helps simple word-counting models; it can hurt models that need grammar or negation.

6 Text normalization: making text consistent

Normalization means cleaning text into one standard, predictable shape — lower-casing it, stripping special characters, and removing stop words. The goal is that "NLP", "nlp", and "Nlp" all end up looking identical to the computer.

☞ The intuition

A model that has only ever seen "Apple" (capitalised) will not automatically know that "apple" means the same thing, unless you first make both look the same. Normalization removes that kind of accidental difference.

✎ Worked example: normalizing one sentence

Take the raw string: *"NLP is AMAZING!!! Isn't it??"*

Step 1. Lower-case everything. `"nlp is amazing!!!  isn't it??"`

Step 2. Strip special characters (keep letters, numbers, apostrophes). `"nlp is amazing isnt it"`

Step 3. Tokenize. `['nlp', 'is', 'amazing', 'isnt', 'it']`

Step 4. Remove stop words (`is`, `it`). `['nlp', 'amazing', 'isnt']`

A shouty, punctuation-heavy sentence became three clean, comparable tokens.

✗ Watch out

Stripping punctuation before checking for stop words can merge two words together if you are not careful (`"isn't"` → `"isnt"`), which then will not match the stop-word list. Always test your pipeline on messy, real text.

✔ Key takeaway

Normalization is housekeeping: it makes two versions of the "same" word look identical before anything else happens.

7 Stemming vs. lemmatization: finding the root

Stemming and lemmatization both try to reduce a word to its root, so that "running", "runs", and "ran" can all be recognised as the same underlying idea. They just get there differently.

☞ The intuition

Stemming is a fast, blunt knife: it chops off endings using fixed rules, even if the result is not a real word. Lemmatization is a slower, careful dictionary lookup: it returns the real base word, called a *lemma*, and it uses grammar (is this word a noun or a verb?) to get there.

★ Everyday picture

Stemming is like abbreviating words on a sticky note without worrying if the abbreviation is a real word — "studying" becomes "studi". Lemmatization is like looking the word up in a dictionary and writing down its proper headword — "studying" becomes "study".

✎ Worked example: stemming vs. lemmatizing four words

Compare the Porter stemmer (a rule-based stemmer) against WordNet lemmatization on four verbs.

Step 1. List the words. `studies`, `studying`, `cries`, `flies`.

Step 2. Apply the Porter stemmer's suffix rules. It strips `-ies` to `-i` and `-ying` to `-i`: `studies`→`studi`, `studying`→`studi`, `cries`→`cri`, `flies`→`fli`.

Step 3. Apply WordNet lemmatization, telling it each word is a verb. It looks up the dictionary form: `studies`→`study`, `studying`→`study`, `cries`→`cry`, `flies`→`fly`.

Step 4. Compare. The stems (`studi`, `cri`, `fli`) are not real English words. The lemmas (`study`, `cry`, `fly`) are.

Both methods correctly group the four words into two root ideas; only the lemmatizer's output is readable on its own.

Where this shows up in ML. Search engines and older text classifiers stem or lemmatize so that a search for "running" also matches documents containing "runs". Modern transformer models skip this step entirely — their subword tokenizer already groups related word forms during training.

✗ Watch out

Stemming can *over-stem* (wrongly merge unrelated words, e.g. "university" and "universe" can both reduce toward "univers") or *under-stem* (fail to merge related words). Lemmatization avoids over-stemming but needs to know the word's part of speech to work correctly.

✔ Key takeaway

Stemming is fast but crude; lemmatization is slower but returns real dictionary words. Pick stemming for speed at scale, lemmatization when the output must be readable.

8 Part-of-speech tagging: labelling grammar roles

Part-of-speech (POS) tagging assigns every word in a sentence a grammatical category: noun, verb, adjective, and so on. This tells later steps *how* a word is being used, not just what the word is.

☞ The intuition

The same word can play different roles: "book" is a noun in "read a book" but a verb in "book a flight". POS tagging looks at the neighbouring words to decide which role applies here.

★ Everyday picture

It is like a casting director reading a script and labelling each character's role — hero, sidekick, villain — based on what they do in the scene, not just their name.

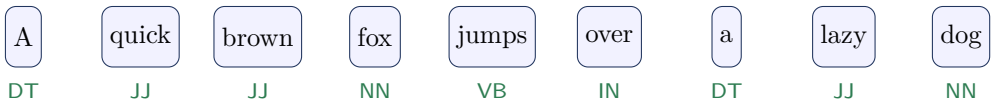


Figure 2: Every token gets one tag directly beneath it: DT = determiner, JJ = adjective, NN = noun, VB = verb, IN = preposition. Reading the tag row tells you the sentence's grammatical skeleton at a glance.

✎ Worked example: tagging "A quick brown fox jumps over a lazy dog"

Step 1. Tokenize. `A, quick, brown, fox, jumps, over, a, lazy, dog`.

Step 2. Tag the first word. `A` introduces a noun phrase without specifying which one → **DT** (determiner).

Step 3. Tag the describing words. `quick` and `brown` describe the coming noun → **JJ** (adjective) each.

Step 4. Tag the subject. `fox` is the thing doing the action → **NN** (noun).

Step 5. Tag the action. `jumps` is what the fox does → **VB** (verb).

Step 6. Tag the connector. `over` links the action to its location → **IN** (preposition).

Step 7. Repeat for the second noun phrase. `a`→DT, `lazy`→JJ, `dog`→NN.

Every one of the nine tokens now carries a grammar label, matching the tag row in the figure above.

Where this shows up in ML. POS tags are a classic feature for parsers and information-extraction systems. Modern taggers are usually small neural sequence models (or a layer inside a larger language model) trained to predict exactly this tag sequence, which is itself a nice first example of a structured-prediction task.

✗ Watch out

A tagger without enough context can guess wrong on ambiguous words — "book" needs the surrounding words to disambiguate noun from verb. Always tag full sentences, never single words in isolation.

✔ Key takeaway

POS tagging is a per-word grammar label, decided using the words around it, not the word alone.

9 Named entity recognition: spotting people, places, and things

Named entity recognition (NER) finds real-world "things" named in text — people, organisations, places, dates, times, and amounts — and labels each with its category.

☞ The intuition

Once you know a word's grammar role (Section 8), NER goes one level further and asks: is this specific noun phrase actually the name of a person, a place, an organisation, or a date?

★ Everyday picture

Imagine highlighting a news article with four colours: yellow for people, green for places, blue for organisations, pink for dates. NER is that highlighting, done automatically.

✎ Worked example: finding entities in one sentence

Take the sentence: *"Abdul Kalam died on July 27, 2015 (07:45 Indian Standard Time GMT+5:30) at Bethany Hospital, Shillong after having collapsed due to cardiac arrest during a lecture he was giving at the Indian Institute of Management (IIM)."*

Step 1. **Tokenize and POS-tag the sentence** (using `word_tokenize` then `pos_tag`, as in Section 8). This gives NER a grammar-labelled token list to search over.

Step 2. **Scan for a person name.** Two capitalised words in a row at the sentence start, acting as the subject → **PERSON**: "Abdul Kalam".

Step 3. **Scan for a date and a time.** "July 27, 2015" matches a date pattern → **DATE**. "07:45 Indian Standard Time GMT+5:30" matches a time pattern → **TIME**.

Step 4. **Scan for locations.** "Shillong" is a single capitalised place name → **GPE** (geo-political entity). "Bethany Hospital" is a capitalised, multi-word facility name → **FACILITY**.

Step 5. **Scan for an organisation.** "Indian Institute of Management (IIM)" is a capitalised multi-word name with a known acronym in parentheses → **ORGANIZATION**.

Five entities recognised, each with its own category, out of one plain sentence.

In NLTK, this whole scan is one short pipeline:

```
from nltk import word_tokenize, pos_tag, ne_chunk
tokens = word_tokenize(text)
pos_tags = pos_tag(tokens)
named_entities = ne_chunk(pos_tags)
```

Where this shows up in ML. NER is the backbone of information extraction: pulling structured facts (who, where, when) out of unstructured text. It is also a key step in retrieval-augmented generation (RAG) pipelines, where recognised entities help link a question to the right facts in a knowledge base.

✗ Watch out

NER depends heavily on capitalisation and known name patterns, so it struggles on all-lowercase text (like casual chat messages) and on names it has never seen before.

✔ Key takeaway

NER labels the real-world "things" in text: people, places, organisations, dates — the raw facts you can pull out and store in a table.

10 Bag of Words: turning text into numbers by counting

Machine-learning models work on numbers, not words. **Bag of Words (BoW)** is the simplest way to turn a document into a number: build one big vocabulary across all documents, then count how many times each vocabulary word appears in each document.

☞ The intuition

Imagine tipping a document into a bag and shaking it: word order is gone, but you can still count how many of each word landed in the bag. That count vector is the document's new "address" in number-space.

★ Everyday picture

It is like a shopping receipt: it does not tell you the order you picked items off the shelf, only how many of each item ended up in your basket.

✎ Worked example: building Bag-of-Words vectors for four sentences

Take this tiny corpus of four documents.

Step 1. Write down the documents. D1: "This is the first document." D2: "This document is the second document." D3: "And this is the third one." D4: "Is this the first document?"

Step 2. Build the vocabulary. Lower-case every word, drop punctuation, collect every distinct word across all four documents, alphabetically: `and, document, first, is, one, second, the, third, this` — 9 words.

Step 3. Count each vocabulary word in each document. This gives one row per document, one column per vocabulary word.

	and	document	first	is	one	second	the	third	this
D1	0	1	1	1	0	0	1	0	1
D2	0	2	0	1	0	1	1	0	1
D3	1	0	0	1	1	0	1	1	1
D4	0	1	1	1	0	0	1	0	1

Notice D1 and D4 produce the identical vector — they use exactly the same set of words, even though D4 is a question. D2 stands out with `document`=2, because it uses that word twice.

Where this shows up in ML. Before deep learning, Bag of Words was the standard input to spam filters and sentiment classifiers: turn each email into a count vector, then train a classifier (like Naive Bayes or logistic regression) on those vectors.

✗ Watch out

BoW throws away word order completely, so "dog bites man" and "man bites dog" produce the exact same vector. It also gives common words like "the" and "is" the same weight as rare, meaningful words — that weighting problem is exactly what TF-IDF (next section) fixes.

✔ Key takeaway

Bag of Words turns a document into a row of word counts against a shared vocabulary. It is simple and fast, but blind to word order and word importance.

11 TF-IDF: weighting words by how much they actually matter

TF-IDF (Term Frequency times Inverse Document Frequency) fixes BoW's main flaw: it downweights words that appear in almost every document (like "document" or "the") and upweights words that are rare across the collection but still show up here.

☞ The intuition

A word that appears in every single document tells you nothing about what makes this document different. A word that appears in only one document is a strong clue about that document's topic. TF-IDF turns that idea into a number.

★ Everyday picture

If every restaurant review says "food", that word is useless for telling reviews apart. But if only one review says "truffle", that word is a strong, distinctive signal — TF-IDF gives "truffle" a much bigger weight than "food".

The formula has two parts, multiplied together:

$$\text{tf-idf}(w, d) = \underbrace{\text{tf}(w, d)}_{\text{how often } w \text{ appears in } d} \times \underbrace{\log\left(\frac{N}{\text{df}(w)}\right)}_{\text{how rare } w \text{ is across all documents}}$$

Here N is the total number of documents, and $\text{df}(w)$ (*document frequency*) is the number of documents that contain the word w at least once.

✎ Worked example: scoring "document" vs. "second" in the same corpus

Reuse the four documents from Section 10 ($N = 4$).

Step 1. Count where "document" appears. It is in D1, D2, and D4, so $\text{df}(\text{document}) = 3$.

Step 2. Compute its idf.

$$\text{idf}(\text{document}) = \log\left(\frac{4}{3}\right) = 0.288.$$

Step 3. Compute its tf-idf in D2, where it appears twice ($\text{tf} = 2$):

$$\text{tf-idf}(\text{document}, D2) = 2 \times 0.288 = 0.575.$$

Step 4. Now do the same for "second". It appears only in D2, so $\text{df}(\text{second}) = 1$, and

$$\text{idf}(\text{second}) = \log\left(\frac{4}{1}\right) = 1.386.$$

With $\text{tf} = 1$ in D2:

$$\text{tf-idf}(\text{second}, D2) = 1 \times 1.386 = 1.386.$$

Step 5. Compare. Even though "document" appears twice and "second" only once, "second" scores over twice as high, because it is rare across the corpus.

The rare word wins the weighting contest — exactly the behaviour Bag of Words could not give us.

✗ Watch out

Real libraries like scikit-learn's `TfidfVectorizer` add a small smoothing constant to the idf formula and then rescale (*normalise*) every document's row to unit length, so the exact numbers it prints will differ slightly from the simplified formula above. The ranking — rare words score higher than common ones — stays the same either way.

Where this shows up in ML. Search engines used TF-IDF for decades to rank which documents best match a query; it is still used today as a fast, strong baseline "retriever" alongside neural

embedding search in modern retrieval-augmented generation systems.

✔ Key takeaway

TF-IDF is Bag-of-Words counts, adjusted so that rare-but-present words count for more than words that are everywhere.

12 Recurrent Neural Networks: giving a model memory

Every method so far treats a document as an unordered bag or a single tagged sentence. A **Recurrent Neural Network (RNN)** is different: it reads a sequence one token at a time and keeps a running memory, called the *hidden state*, that carries information from earlier tokens forward.

☞ The intuition

A plain (feedforward) network looks at its whole input at once and has no memory between inputs. An RNN reads word by word, and after each word it updates a small memory that summarises everything it has read so far — so word 10 can still be influenced by word 1.

★ Everyday picture

Reading a novel one page at a time, you do not forget who the villain is just because you turned the page. Your mental summary of the story so far is exactly the RNN's hidden state.

The hidden state update, at each time step t, is:

$$h_t = \tanh(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b)$$

Here x_t is the input at step t (e.g. the current word's embedding), h_{t-1} is the memory carried over from the previous step, W_{xh} and W_{hh} are weights the network learns, b is a bias term, and tanh is a squashing function that keeps the hidden state's values between -1 and 1.

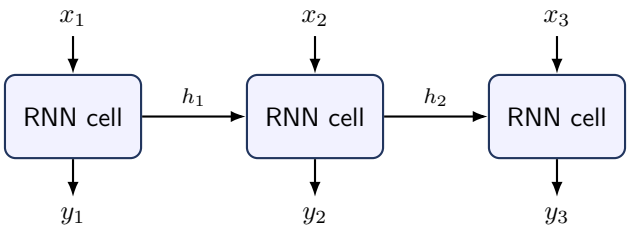


Figure 3: The same RNN cell "unfolded" across three time steps. Each cell reads its own input x_t and the hidden state handed over from the step before — that handed-over arrow is the network's memory.

✎ Worked example: one hidden state update by hand

Use a toy RNN with a single input and a single hidden unit, so every number is scalar. Let $x_1 = 1$, $x_2 = 0.5$, $h_0 = 0$ (no memory yet), $W_{xh} = 0.5$, $W_{hh} = 0.8$, $b = 0.1$.

Step 1. Write down what we know. All five numbers above; we want h_1 and h_2.

Step 2. Compute h_1. Plug into the formula:

$$h_1 = \tanh(0.5 \times 1 + 0.8 \times 0 + 0.1) = \tanh(0.6) = 0.537.$$

Step 3. Compute h_2, using h_1 this time instead of 0.

$$h_2 = \tanh(0.5 \times 0.5 + 0.8 \times 0.537 + 0.1) = \tanh(0.780) = 0.652.$$

Step 4. Sense-check. Both hidden states sit inside $(-1, 1)$, as tanh guarantees. And h_2 was computed *using* h_1 — so information from x_1 is still shaping the network's output at step 2.

That carry-forward from h_1 into the h_2 calculation is memory, made concrete in numbers.

RNNs solve two kinds of problems particularly well:

RNN application	What the sequence is
Speech recognition	audio samples over time
Music generation	notes over time
Machine translation	words in the source language, in order
Video action analysis	video frames over time
DNA / genome sequence study	bases along a strand

Where this shows up in ML. RNNs were the standard way to model sequences — text, speech, time series — before Transformers. Their descendants, LSTMs and GRUs, added extra gates to fix the weakness below; today, Transformers have mostly replaced RNNs for language, but RNNs still see use in smaller, resource-constrained, or strictly-sequential settings.

✗ Watch out

Plain RNNs struggle with *long-term dependencies*: information from many steps back keeps getting multiplied by the same weight, so it either shrinks toward zero (the *vanishing gradient* problem) or blows up, before it ever reaches a distant later step. This is exactly why LSTMs and GRUs were invented.

✔ Key takeaway

An RNN reads a sequence one item at a time and carries a hidden state forward as memory — but a plain RNN's memory of distant, early inputs fades fast.

Glossary & symbols

Key terms.

token	one small piece of text — usually a word or a punctuation mark — produced by tokenization.
corpus	a collection of documents used for training or testing an NLP tool.
stop word	a very common word (*the*, *is*, *to*) that is often removed before counting words.
stem	the crude root a stemmer returns; may not be a real word.
lemma	the real dictionary root a lemmatizer returns.
POS tag	a grammar label (noun, verb, adjective, . . .) attached to one token.
named entity	a real-world thing named in text: a person, place, organisation, date, or amount.
vocabulary	the full list of distinct words collected across a corpus, used as the columns of a Bag-of-Words table.

document frequency (df) the number of documents in a corpus that contain a given word at least once.

hidden state the running memory an RNN carries forward from one time step to the next.

Symbols at a glance.

Symbol	Meaning
N	total number of documents in the corpus
$\mathrm{tf}(w, d)$	how many times word w appears in document d
$\mathrm{df}(w)$	number of documents that contain word w at least once
x_t	the RNN's input (e.g. a word) at time step t
h_t	the RNN's hidden state (memory) after time step t
W_{xh}, W_{hh}, b	the RNN's learned weights and bias term
tanh	a squashing function that keeps a value between -1 and 1

Further reading

- Steven Bird, Ewan Klein, and Edward Loper, *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit* — practical, code-first coverage of everything in this chapter and beyond.
- The Hugging Face Transformers documentation — tutorials and code for running pre-trained models such as BERT and GPT on real tasks.
- Search Google Scholar for papers citing the original GPT-2 paper by Alec Radford et al., to see how sequence modelling moved from RNNs to Transformers.